Part B: Structural Aspects (UML Class Diagram)

Key Classes with Attributes and Methods

1. Pipeline

Represents a CI/CD pipeline execution instance.

Attributes:

- - pipelineId: string (private)
- - pipelineName: string (private)
- - status: PipelineStatus (private)
- - startTime: DateTime (private)
- - endTime: DateTime (private)
- - buildNumber: int (private)
- - branch: string (private)
- - triggeredBy: string (private)

Methods:

- + Pipeline(name: string, branch: string) (public constructor)
- + start(): void (public)
- + stop(): void (public)
- + getStatus(): PipelineStatus (public)
- + getDuration(): long (public)
- + toJson(): string (public)
- - updateStatus(status: PipelineStatus): void (private)
- - validateConfiguration(): bool (private)

2. FailureDetector

Monitors pipelines and detects failures.

Attributes:

- - detectorId: string (private)
- - thresholds: Map<string, double> (private)
- - monitoringInterval: int (private)
- - isActive: bool (private)
- # lastCheckTime: DateTime (protected)

Methods:

- + FailureDetector(interval: int) (public constructor)
- + detectFailure(pipeline: Pipeline): Failure (public)
- + startMonitoring(): void (public)
- + stopMonitoring(): void (public)
- + setThreshold(metric: string, value: double): void (public)
- # analyzeMetrics(logs: LogData): bool (protected)
- # classifyFailureType(error: string): FailureType (protected)
- - parseErrorLogs(logs: string): ErrorInfo (private)

3. Failure

Represents a detected pipeline failure.

Attributes:

- - failureId: string (private)
- - failureType: FailureType (private)
- - errorMessage: string (private)
- - stackTrace: string (private)
- - timestamp: DateTime (private)
- - severity: SeverityLevel (private)
- - affectedStage: string (private)
- - pipelineRef: Pipeline (private)

Methods:

- + Failure(type: FailureType, message: string, pipeline: Pipeline) (public)
- + getFailureType(): FailureType (public)
- + getSeverity(): SeverityLevel (public)
- + getErrorMessage(): string (public)
- + toString(): string (public)
- - calculateSeverity(): SeverityLevel (private)
- - extractStackTrace(error: Exception): string (private)

4. RootCauseAnalyzer

Analyzes failures to determine root causes.

Attributes:

- - analyzerId: string (private)
- - knowledgeBase: KnowledgeBase (private)
- - analysisRules: List<AnalysisRule> (private)
- # diagnosticCache: Map<string, Diagnosis> (protected)

Methods:

- + RootCauseAnalyzer(kb: KnowledgeBase) (public)
- + analyze(failure: Failure): Diagnosis (public)
- + addAnalysisRule(rule: AnalysisRule): void (public)
- # applyRules(failure: Failure): List<Cause> (protected)
- # scoreRootCauses(causes: List<Cause>): Cause (protected)
- - matchPatterns(error: string): List<Pattern> (private)
- - consultKnowledgeBase(pattern: Pattern): Cause (private)

5. RecoveryStrategy (Abstract)

Abstract base class for recovery strategies.

Attributes:

- # strategyName: string (protected)
- # priority: int (protected)
- # maxRetries: int (protected)
- # successRate: double (protected)

Methods:

- + RecoveryStrategy(name: string, priority: int) (public)
- + execute(failure: Failure): RecoveryResult (public, abstract)
- + canHandle(failure: Failure): bool (public, abstract)
- + getPriority(): int (public)
- # validatePreconditions(failure: Failure): bool (protected)
- # recordResult(result: RecoveryResult): void (protected)

6. RetryStrategy (extends RecoveryStrategy)

Implements retry-based recovery.

Attributes:

- - retryCount: int (private)
- - backoffMultiplier: double (private)
- - maxWaitTime: int (private)

Methods:

- + RetryStrategy(maxRetries: int) (public)
- + execute(failure: Failure): RecoveryResult (public, override)
- + canHandle(failure: Failure): bool (public, override)
- - calculateBackoff(attempt: int): int (private)
- - retryPipeline(pipeline: Pipeline): bool (private)

7. ConfigFixStrategy (extends RecoveryStrategy)

Fixes configuration-related failures.

Attributes:

- - configPatterns: Map<string, string> (private)
- - backupConfigs: List<Configuration> (private)

Methods:

- + ConfigFixStrategy() (public)
- + execute(failure: Failure): RecoveryResult (public, override)
- + canHandle(failure: Failure): bool (public, override)
- - identifyConfigIssue(failure: Failure): ConfigIssue (private)
- - applyFix(issue: ConfigIssue): bool (private)
- - rollbackConfig(backup: Configuration): void (private)

8. RecoveryEngine

Orchestrates the recovery process.

Attributes:

- - engineId: string (private)
- - strategies: List<RecoveryStrategy> (private)
- - activeRecoveries: Map<string, Recovery> (private)
- - maxConcurrentRecoveries: int (private)
- # executor: ThreadPool (protected)

Methods:

- + RecoveryEngine(maxConcurrent: int) (public)
- + recoverFromFailure(failure: Failure, diagnosis: Diagnosis): RecoveryResult (public)
- + registerStrategy(strategy: RecoveryStrategy): void (public)
- + getActiveRecoveries(): List<Recovery> (public)
- - selectBestStrategy(failure: Failure, diagnosis: Diagnosis): RecoveryStrategy (private)
- - executeAsync(strategy: RecoveryStrategy, failure: Failure): Future<RecoveryResult> (private)
- # monitorRecoveryProgress(recovery: Recovery): void (protected)

9. KnowledgeBase

Stores historical data and learned patterns.

Attributes:

- - failurePatterns: Map<string, Pattern> (private)
- - recoveryHistory: List<RecoveryRecord> (private)
- - mlModel: MachineLearningModel (private)
- - databaseConnection: Connection (private)

Methods:

- + KnowledgeBase(dbConnection: Connection) (public)
- + learnFromRecovery(recovery: Recovery): void (public)
- + findSimilarFailures(failure: Failure): List<Failure> (public)
- + predictRecoverySuccess(strategy: RecoveryStrategy, failure: Failure): double (public)
- + updatePattern(pattern: Pattern): void (public)
- - trainModel(data: List<RecoveryRecord>): void (private)
- - extractFeatures(failure: Failure): FeatureVector (private)

10. NotificationService

Sends notifications to developers and admins.

Attributes:

- - channels: List<NotificationChannel> (private)
- - templates: Map<string, Template> (private)
- - notificationQueue: Queue<Notification> (private)

Methods:

- + NotificationService() (public)
- + sendFailureAlert(failure: Failure, recipients: List<User>): void (public)
- + sendRecoveryUpdate(recovery: Recovery, recipients: List<User>): void (public)
- + addChannel(channel: NotificationChannel): void (public)
- - formatMessage(template: Template, data: Map): string (private)
- - deliverNotification(notification: Notification): bool (private)

UML Class Diagram with Relationships

The following UML class diagram shows the relationships between all classes, including inheritance, association, aggregation, and composition relationships.

Relationships Summary:

Inheritance:

- RetryStrategy —▷ RecoveryStrategy
- ConfigFixStrategy —▷ RecoveryStrategy

Composition (strong ownership):

- RecoveryEngine ◆—→ RecoveryStrategy (1 to 0..*)
- Pipeline ◆—→ Failure (1 to 0..*)
- Failure ◆—→ Diagnosis (1 to 1)

Aggregation (weak ownership):

- RootCauseAnalyzer ◇—→ KnowledgeBase (1 to 1)
- RecoveryEngine ◇—→ NotificationService (1 to 1)

Association (uses):

- FailureDetector —→ Pipeline (monitors)
- FailureDetector —→ Failure (creates)
- RootCauseAnalyzer —→ Failure (analyzes)
- RecoveryEngine —→ Failure (recovers from)
- RecoveryStrategy —→ Failure (handles)
- KnowledgeBase —→ RecoveryStrategy (informs)
- NotificationService —→ Failure (notifies about)

Cardinality Specifications:

- • RecoveryEngine has 0..* RecoveryStrategy (zero or many strategies)
- • Pipeline can have 0..* Failures (zero or many failures)
- • Each Failure has exactly 1 Diagnosis (one-to-one)
- • RootCauseAnalyzer uses exactly 1 KnowledgeBase (one-to-one)
- • RecoveryEngine uses exactly 1 NotificationService (one-to-one)
- • FailureDetector monitors 1..* Pipelines (one or many)
- • RecoveryStrategy can handle 0..* Failures (zero or many)